

LAPORAN TUGAS AKHIR UAS

Design and Implementation of an Intelligent System-Based Robotic System



Disusun Oleh (Kelompok UAS):

Nama	NIM
Muhammad Khalis Farhan Azvi	— 22/493199/TK/54016
Valen Fatli	— 22/493755/TK/54109
Yasir Abdulaziz	— 22/497079/TK/54468
Galih Salman Sadewo	— 22/498121/TK/54625
Faris Afif Ridyasmara	— 22/502798/TK/54902

Course Lecturer: Dr.Eng. Dwi Joko Suroso, S.T., M.Eng.

Robotics Instrumentation Systems

Departemen of Nuclear Engineering and Engineering Physics

Faculty of Engineering

Universitas Gadjah Mada

Tahun 2026

Contents

1	Introduction	1
2	Literature Review	4
3	System Design	5
3.1	Hardware Architecture	5
3.1.1	Processing and Control Unit	5
3.1.2	Sensing System	6
3.1.3	Actuation and Kinematic Control	6
3.1.4	Power Distribution Topology	7
3.2	Visual System Design (Visual)	8
3.2.1	Image Acquisition and Camera Management (<i>camera_manager.py</i>)	8
3.2.2	Preprocessing and Color Segmentation (<i>image_processor.py</i>) . . .	10
3.2.3	Object Contour Detection (<i>object_detector.py</i>)	12
3.2.4	Visual Distance Estimation (<i>distance_estimator.py</i>)	13
3.2.5	Service Integration and Calibration (<i>calibration_service.py</i> & <i>stream_service.py</i>)	15
3.3	Control Logic and Navigation (Logic)	17
3.3.1	Motor Driver Controller (<i>motor_manager.py</i>)	18
3.3.2	Ultrasonic Sensor Manager (<i>ultrasonic_manager.py</i>)	20
3.3.3	PID Control Algorithm and Main Navigation (<i>navigator.py</i>) . . .	22
4	Experimental Results and Discussion	30
4.1	DC Motor Speed Calibration and Path Alignment	30
4.2	Ultrasonic Sensor Accuracy and Fluctuation Testing	30
4.3	Sequential Navigation Performance Analysis on the Track	31
4.4	Obstacle Evasion and Physical Track Boundary Challenges	32
4.5	Webcam Color Stability and Exposure Constraints	32
5	Conclusion and Recommendations	33

5.1	Conclusion	33
5.2	Recommendations	33

List of Figures

1	Block diagram of the hardware architecture and control signal flow. . . .	5
2	Physical hardware implementation on the autonomous color-tracking robot chassis.	8
3	Robotic Navigation Test Track Layout.	31

List of Tables

1	HC-SR04 Ultrasonic Distance Sensor Accuracy Test Data.	30
---	--	----

1 Introduction

Autonomous mobile robots are increasingly used in indoor applications such as material transportation, inspection, surveillance, object retrieval, healthcare, and educational robotics. To operate autonomously, a mobile robot must be able to perceive its surroundings, identify a desired destination, determine an appropriate direction of motion, and avoid collisions with nearby objects. These functions depend strongly on the sensing system because environmental information is required for object recognition, distance measurement, motion planning, and robot control [1].

Different sensing technologies can be used to support autonomous navigation, including cameras, ultrasonic sensors, infrared sensors, LiDAR, and depth cameras. A monocular camera is particularly attractive for low-cost robots because it can provide information regarding object color, apparent size, shape, and position within the robot's field of view. Recent studies have demonstrated that monocular vision can be used to recognize obstacles and produce navigation commands for mobile robots operating in indoor environments [2]. Vision-based systems can also perform real-time obstacle detection and path generation without relying on expensive LiDAR or depth-sensing devices [3].

Despite these advantages, a monocular camera does not directly measure physical distance. The apparent dimensions of an object in an image depend on its actual dimensions, camera parameters, viewing angle, and distance from the camera. Therefore, the position and dimensions of a bounding box can indicate the relative direction and proximity of an object, but additional calibration is required to convert these visual features into an estimated physical distance.

Color-based object detection also presents practical challenges. Changes in illumination, shadows, reflections, and camera exposure can shift the pixel values representing the same physical color. Consequently, permanently defined color thresholds may not provide consistent detection under different environmental conditions. A recent target-recognition study using monocular vision showed that HSV-based segmentation and threshold calibration under the actual experimental illumination can improve the separation of a colored target from the background [4]. This approach is particularly relevant when the target color is only specified immediately before the robot begins its task.

Ultrasonic sensing provides a low-cost method for obtaining direct distance measurements. A single ultrasonic sensor can be used to detect frontal obstacles, determine whether an object has entered a critical range, and support collision-avoidance decisions [5]. However, ultrasonic sensing alone cannot reliably determine the color or visual identity of an object. Its measurement is also limited by the sensor's field of view, object

orientation, and surface characteristics. Therefore, camera and ultrasonic measurements provide complementary information: the camera identifies and localizes objects in the image, while the ultrasonic sensor supplies frontal distance information.

Existing vision-based navigation studies commonly employ convolutional neural networks, semantic segmentation, geometric models, or mapping algorithms [2, 3]. Although these methods can provide accurate environmental perception, they may require considerable computational resources, training data, or additional system complexity. Such requirements can limit their implementation on embedded platforms such as the Raspberry Pi 3 Model B. Furthermore, many systems assume that the target characteristics are known before deployment, whereas the target color in the considered competition is determined at the test location.

This study develops a two-wheeled mobile robot capable of autonomously navigating toward a color-defined target while avoiding obstacles on a structured indoor track. The robot uses a webcam as the primary visual sensor, an ultrasonic distance sensor as the frontal distance sensor, and a Raspberry Pi 3 Model B as the processing and control unit. Image acquisition, color segmentation, contour extraction, and bounding-box generation are implemented using the OpenCV library.

Before autonomous navigation begins, the target color is calibrated under the actual lighting conditions of the competition environment. The calibrated color thresholds are then used to segment the target and relevant obstacle regions from the camera frames. The detected regions are represented by bounding boxes. The horizontal center of each bounding box is used to determine whether an object is located on the left, center, or right side of the robot's field of view.

The dimensions of the bounding box are used as visual features for estimating object proximity. During the distance-calibration procedure, the bounding-box measurements are paired with reference distance measurements obtained from the HC-SR04 sensor. This procedure establishes a relationship between the apparent dimensions of the detected object and its physical distance from the robot. During navigation, the ultrasonic sensor also provides an additional safety measurement when an obstacle is located directly in front of the robot.

Based on the detected target position, the estimated distance and the obstacle information, the navigation algorithm determines the required motion command. The available commands include moving forward, turning left, turning right, stopping, and performing an obstacle-avoidance maneuver. After avoiding an obstacle, the robot redirects its movement toward the detected target until it reaches the designated target area.

The objective of this study is to design, implement and evaluate an integrated perception and navigation system that enables a two-wheeled mobile robot to reach a color-defined target without colliding with obstacles. The main contributions of this work are summarized as follows:

1. On-site color-calibration procedure that enables the robot to adapt to a target color specified immediately before operation;
2. Lightweight OpenCV-based object-detection method that represents detected targets and obstacles using bounding boxes;
3. Ultrasonic-assisted calibration procedure that relates bounding-box dimensions to physical object distance;
4. Integrated navigation strategy that combines target-directed movement and reactive obstacle avoidance; and
5. Implementation using a webcam, an ultrasonic distance sensor, and a Raspberry Pi 3 Model B as a low-cost embedded robotic platform.

The proposed system emphasizes computationally lightweight image processing and low-cost sensing instead of computationally intensive perception models or expensive depth sensors. Its performance is evaluated on a structured indoor track considering target and obstacle detection accuracy, distance-estimation error, maneuverability, and the success rate of reaching target color during the experiment.

2 Literature Review

Autonomous navigation requires robust obstacle avoidance mechanisms, which have been widely addressed in recent literature through various advanced control strategies. For instance, Yan et al. propose a multi-layered approach for unmanned vehicles that combines the A* algorithm for global static path planning with the Velocity-Obstacle (VO) method to dynamically evade moving targets [6]. This highlights the necessity of hybrid, global-local systems to ensure navigational safety in complex dynamic environments.

Other studies focus on optimizing real-time reactive avoidance models. Zhang et al. introduce an improved Artificial Potential Field (APF) method that incorporates the velocity information of moving obstacles to prevent unreasonably long evasion paths, while also solving the unreachable target problem common in traditional APF algorithms [7]. Furthermore, for high-speed driving scenarios, Kim et al. utilize a Real-Time Output Constraints Model Predictive Control (RCMPC) algorithm, which optimizes the avoidance path to significantly reduce computation time while ensuring safety and smooth maneuverability [8].

Building upon these foundational concepts of real-time perception and reactive control, the robotic system described in the main project report implements an efficient, low-cost alternative for indoor autonomous navigation. Rather than relying on computationally heavy path-planning algorithms, it successfully integrates a monocular webcam for color-based target tracking and an ultrasonic sensor for direct frontal obstacle detection. By feeding this sensory data into a proportional-integral-derivative (PID) controller and a sequential finite state machine, the robot can dynamically pursue its target and reactively steer clear of obstacles with minimal computational overhead.

3 System Design

This chapter describes the comprehensive robotic system design architecture, which is divided into hardware configuration, computer vision subsystem, and navigation control logic.

3.1 Hardware Architecture

The hardware architecture of the autonomous color-tracking robot is designed as an integrated embedded robotic platform. Conceptually, this system is divided into four main functional blocks: the processing and control unit, the sensing system, the actuation mechanism, and the power distribution topology. The data flow, control signals, and power distribution among these functional blocks are represented by the high-level architecture diagram in Figure 1.

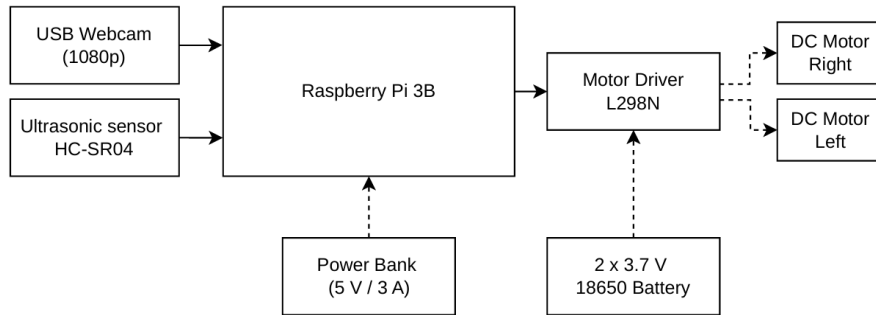


Figure 1: Block diagram of the hardware architecture and control signal flow.

3.1.1 Processing and Control Unit

The core computational and control operations of the entire system rely on a Raspberry Pi 3 Model B Single-Board Computer (SBC). The use of this SBC is essential to provide the considerable computational resources required to execute the OpenCV-based image processing algorithms and bounding-box generation without relying on external computing. The Raspberry Pi acts as the main processing and control unit that reads the continuous visual data stream, processes the navigation decision logic, and translates these calculations into Pulse Width Modulation (PWM) control signals. These signals are subsequently distributed through the General Purpose Input/Output (GPIO) pins to dictate the responses of the actuation system.

3.1.2 Sensing System

The robot's ability to perceive its surroundings and gather environmental information is realized through the integration of two distinct sensing technologies:

- **Monocular Camera (1080p USB Webcam):** Serving as the primary visual sensor, the webcam captures light reflections from objects and converts them into RGB pixel matrices. These raw data matrices are streamed to the processing unit to be transformed into the HSV color space. This mapping is fundamental for the vision-based system, as it separates pure chromatic information (Hue) from environmental illumination intensity (Value). This enhances the robustness of the HSV-based segmentation, allowing the robot to identify the color-defined target and extract bounding boxes even under varying lighting conditions.
- **Ultrasonic Sensor (HC-SR04):** Operating based on the Time-of-Flight (ToF) principle of mechanical waves, this sensor emits 40 kHz ultrasonic pulses and calculates the time elapsed until the echo is received. It is implemented as a frontal distance sensor to provide direct physical distance measurements and support reactive collision-avoidance decisions. If a frontal obstacle is detected within a critical proximity, the sensor input triggers an interrupt in the main control logic to immediately halt traction, preventing physical collisions when the visual sensor is occluded or experiences response latency.

3.1.3 Actuation and Kinematic Control

The conversion of electrical control signals into mechanical torque is managed by a differential drive configuration, suitable for a two-wheeled mobile robot.

- **L298N Motor Driver Circuit:** This module functions as a power interface bridge between the Raspberry Pi's low-voltage GPIO pins (3.3V) and the inductive load of the motors, which demands high current supply. Operating on a Dual H-Bridge circuit topology, the L298N allows dynamic voltage polarity reversal to control the direction of motion, while simultaneously switching the PWM signals from the processing unit to regulate the duty cycle (rotational speed).
- **Direct Current (DC) Motors:** The final motion execution relies on two DC motors coupled to the left and right wheels of the chassis. By asynchronously varying the rotational speed distribution between the two wheels, the robot can perform skid-steering to smoothly execute required motion commands, including moving

forward, turning left, turning right, stopping, and performing obstacle-avoidance maneuvers.

3.1.4 Power Distribution Topology

The computational stability of the embedded platform heavily depends on the implemented power supply scheme. Because DC motors are inductive loads—prone to generating inrush currents upon starting and back-EMF—the system employs a split power supply topology with an integrated common ground.

- **Logic Power Supply:** The Raspberry Pi is exclusively powered by a closed power source (5V/3A Powerbank). Isolating this current source is crucial to maintain computational voltage stability and prevent transient voltage drops that could reset the operating system kernel when the actuators operate at maximum capacity.
- **Mechanical Load Power Supply:** The L298N driver circuit and the DC motors receive power from a series configuration of two 18650 Lithium-ion batteries. The 7.4V nominal voltage from this array is injected directly into the driver's input terminals. Although the power sources are physically separated, the ground terminals (GND) of the 18650 battery circuit and the Raspberry Pi's GPIO are strictly unified. This common ground ensures that all PWM control pulses share a uniform zero-equipotential reference, guaranteeing that the operational logic is translated precisely without noise interference.

The physical implementation of these integrated hardware components on the mechanical chassis is shown in Figure 2. This concrete integration of functional blocks ultimately enables the mobile robot to acquire actual environmental parameters, execute lightweight object-detection methods, and perform precise kinematic maneuvers.

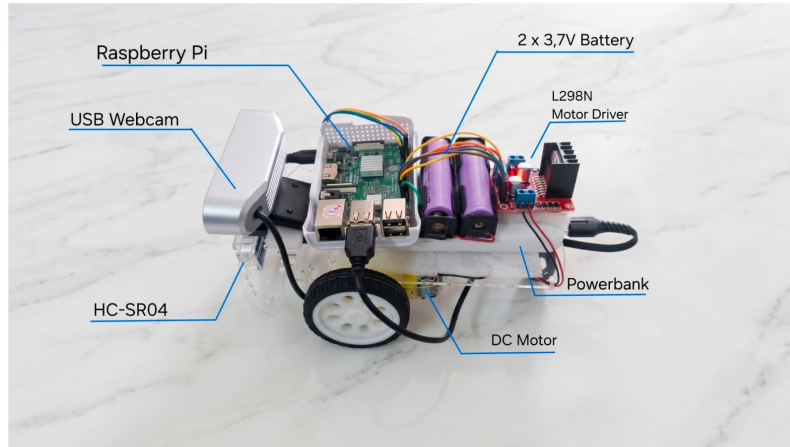


Figure 2: Physical hardware implementation on the autonomous color-tracking robot chassis.

3.2 Visual System Design (Visual)

The design of the visual subsystem focuses on the real-time processing of digital images captured by a USB Webcam. The camera is positioned at the front of the robot to capture the environmental conditions directly. The processing pipeline of this visual system is divided into four main stages: image acquisition, preprocessing and color segmentation (masking), object contour detection, and visual distance estimation.

3.2.1 Image Acquisition and Camera Management (*camera_manager.py*)

Raw images are obtained using a USB Webcam with a native resolution of 320×240 pixels. The video capture format is set using MJPG (*Motion JPEG*) compression via the V4L2 (*Video for Linux 2*) driver to minimize USB bandwidth usage and the processing load on the Raspberry Pi.

To avoid bottlenecks in the execution of the main program, the camera frame reading process is isolated into a separate background thread that runs asynchronously with a sleep interval of 0.03 seconds between frame captures. This ensures that the latest frame is always available and ready to be accessed by the image processing module at any time without blocking the main execution thread. The camera management implementation code can be seen in Listing 1.

```
1 import cv2
2 import threading
3 import time
4 import numpy as np
```

```

5
6 class CameraManager:
7     def __init__(self, src=0):
8         self.src = src
9         # Use V4L2 backend which is much more stable on Raspberry Pi/
Linux
10        self.stream = cv2.VideoCapture(self.src, cv2.CAP_V4L2)
11
12        # Set format to MJPG to save USB bandwidth on Raspberry Pi
13        self.stream.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'
MJPG'))
14        self.stream.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
15        self.stream.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
16
17        self.grabbed = False
18        # Initialize with a black frame so it is not None
19        self.frame = np.zeros((240, 320, 3), dtype=np.uint8)
20        self.stopped = False
21
22        if self.stream.isOpened():
23            print(f"[CAM] Camera {self.src} detected.")
24            # Capture one initial frame synchronously
25            self.grabbed, self.frame = self.stream.read()
26        else:
27            print(f"[CAM] [ERROR] Camera {self.src} not found!")
28
29        def start(self):
30            # Run frame reading in the background thread
31            threading.Thread(target=self.update, args=(), daemon=True).
start()
32            return self
33
34        def update(self):
35            while not self.stopped:
36                if not self.stream.isOpened():
37                    break
38                self.grabbed, self.frame = self.stream.read()
39                # Small delay to not overload the CPU
40                time.sleep(0.03)
41
42        def read(self):
43            return self.frame
44
45        def stop(self):
46            self.stopped = True

```

```

47     self.stream.release()
48
49     def switch_source(self, new_src):
50         """Dynamically switch camera input"""
51         self.stopped = True
52         time.sleep(0.1) # Give a small delay for the update thread to
53         finish
54         self.stream.release()
55
56         self.src = new_src
57         self.stream = cv2.VideoCapture(self.src, cv2.CAP_V4L2)
58         self.stream.set(cv2.CAP_PROP_FOURCC, cv2.VideoWriter_fourcc(*'
59         MJPG'))
60
61         self.stream.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
62         self.stream.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
63
64
65         self.grabbed = False
66         self.frame = np.zeros((240, 320, 3), dtype=np.uint8)
67         self.stopped = False
68
69
70         if self.stream.isOpened():
71             print(f"[CAM] Successfully switched to Camera {self.src}")
72             self.grabbed, self.frame = self.stream.read()
73             self.start()
74             return True
75         else:
76             print(f"[CAM] [ERROR] Failed to open Camera {self.src}")
77             # Restart the update thread so the program does not hang
78             self.start()
79             return False

```

Listing 1: Camera Manager Module Code (camera_manager.py)

3.2.2 Preprocessing and Color Segmentation (*image_processor.py*)

Before color segmentation is performed, the image frame, which is of type BGR (*Blue-Green-Red*), is first filtered using a Gaussian Blur filter with a kernel size of 11×11 pixels. This process aims to reduce high-frequency noise and smooth the edges of objects. Subsequently, the image color space is converted from BGR to HSV (*Hue, Saturation, Value*) because the HSV color space is more stable against fluctuations in environmental light intensity.

Object segmentation in the image processing module is carried out by dividing the color thresholding results into two main binary masks:

1. **Target Mask (*mask_target*):** This mask isolates the target color based on the previously calibrated HSV color threshold values. The threshold range is defined by a lower bound (HSV_{lower}) and an upper bound (HSV_{upper}). To remove isolated small binary pixel noise, a morphological operation consisting of erosion (*erode*) with 2 iterations, followed by dilation (*dilate*) with 2 iterations, is applied.
2. **Vivid Color Mask (*mask_colorful*):** This mask is used to detect all objects in the environment that have a vivid color saturation (such as origami paper or other physical markers). The lower threshold parameters are set at $HSV = [0, 100, 80]$ and the upper threshold at $HSV = [180, 255, 255]$.
3. **Obstacle Mask (*mask_obstacle*):** An object is considered an obstacle if it has vivid color characteristics but its color is not the target color. The obstacle mask is obtained through a bitwise AND logic operation between the vivid color mask and the negation of the target mask:

$$mask_obstacle = mask_colorful \wedge \neg(mask_target) \quad (1)$$

The implementation code for color segmentation can be seen in Listing 2.

```

1 import cv2
2 import numpy as np
3
4 class ImageProcessor:
5     def process_masks(self, frame, lower_color, upper_color):
6         blurred = cv2.GaussianBlur(frame, (11, 11), 0)
7         hsv = cv2.cvtColor(blurred, cv2.COLOR_BGR2HSV)
8
9         # 1. Mask for Target (According to user click color)
10        mask_target = cv2.inRange(hsv, lower_color, upper_color)
11        mask_target = cv2.erode(mask_target, None, iterations=2)
12        mask_target = cv2.dilate(mask_target, None, iterations=2)
13
14        # 2. Mask for Vividly Colored Objects (Origami paper/binder)
15        # Colorful paper usually has high Saturation and Value (> 100)
16        lower_colorful = np.array([0, 100, 80])
17        upper_colorful = np.array([180, 255, 255])
18        mask_colorful = cv2.inRange(hsv, lower_colorful, upper_colorful
19        )
20
21        mask_colorful = cv2.erode(mask_colorful, None, iterations=2)
22        mask_colorful = cv2.dilate(mask_colorful, None, iterations=2)

```

```

23     # 3. Obstacle Mask
24     # An object is considered a Distractor IF its color is vivid,
    BUT NOT the target color
25     mask_obstacle = cv2.bitwise_and(mask_colorful, cv2.bitwise_not(
    mask_target))
26
27     return mask_target, mask_obstacle

```

Listing 2: Image Processor Module Code (image_processor.py)

3.2.3 Object Contour Detection (*object_detector.py*)

Once the binary masks (*mask_target* and *mask_obstacle*) are formed, the physical coordinates and dimensions of the objects are identified using an external contour retrieval algorithm (`cv2.findContours` with the `CHAIN_APPROX_SIMPLE` approximation method). The detected contours are filtered based on their pixel area. Contours with an area less than the minimum threshold (`min_area = 500` pixels) will be ignored to filter out dust particles or minor background noise.

Each valid contour that passes the filtering is enclosed in an upright bounding box (*Bounding Box*) defined by the top-left pixel coordinates (x, y) and the width (w) and height (h) dimensions using the `cv2.boundingRect` function. The list of detected objects is then sorted by the bounding box area ($w \times h$) in descending order so that the largest object always occupies the first index. The object detection implementation code can be seen in Listing 3.

```

1  import cv2
2
3  class ObjectDetector:
4      def __init__(self, min_area=500):
5          # min_area is the minimum pixel size to prevent objects from
    being considered dust/noise
6          self.min_area = min_area
7
8      def find_objects(self, mask):
9          # Find contours (outer outlines of white objects) on the mask
10         contours, _ = cv2.findContours(mask.copy(), cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
11
12         valid_objects = []
13         for contour in contours:
14             area = cv2.contourArea(contour)
15             # Ignore objects that are too small

```

```

16         if area > self.min_area:
17             # Get x, y coordinates, width (w), and height (h)
18             x, y, w, h = cv2.boundingRect(contour)
19             valid_objects.append((x, y, w, h))
20
21         # Sort objects based on bounding box area (largest to smallest)
22         valid_objects = sorted(valid_objects, key=lambda obj: obj[2] *
23                                obj[3], reverse=True)
24         return valid_objects

```

Listing 3: Object Detector Module Code (object_detector.py)

3.2.4 Visual Distance Estimation (*distance_estimator.py*)

The vision system is also equipped with the ability to estimate the physical distance of objects from the camera utilizing the pinhole camera perspective projection model (*pinhole camera model*). The relationship between the pixel size of the object on the camera sensor and its physical distance is inversely proportional. The physical distance of the object (D) is estimated using the following mathematical equation:

$$D = \frac{W_{\text{ref}} \times D_{\text{ref}}}{w} \quad (2)$$

where:

- D is the estimated physical distance of the object from the camera (in cm).
- W_{ref} is the width of the reference object in pixels in the image when the calibration process was performed.
- D_{ref} is the actual physical reference distance from the camera to the object when the calibration process was performed (in cm).
- w is the width dimension of the object's bounding box in pixels in the current frame.

The initial calibration values, namely W_{ref} and D_{ref} , are stored persistently in the configuration file `kalibrasi_jarak.txt` so that the system can use them immediately when restarted. The distance estimation implementation code can be seen in Listing 4.

```

1 import os
2
3 class DistanceEstimator:
4     def __init__(self, file_path="kalibrasi_jarak.txt"):
5         self.file_path = file_path

```

```

6         self.reference_pixel_width = 0.0
7         self.known_distance = 0.0
8         self.is_calibrated = False
9         self.load_calibration()
10
11     def load_calibration(self):
12         if os.path.exists(self.file_path):
13             try:
14                 with open(self.file_path, "r") as f:
15                     lines = f.readlines()
16                     if len(lines) >= 2:
17                         self.reference_pixel_width = float(lines[0].
strip())
18                         self.known_distance = float(lines[1].strip())
19                         self.is_calibrated = True
20                         print(f"[DISTANCE] Calibration loaded: {self.
reference_pixel_width}px at {self.known_distance}cm")
21             except Exception as e:
22                 print(f"[DISTANCE] Failed to load calibration file: {e}
")
23
24     def calibrate(self, known_distance, pixel_width):
25         if known_distance > 0 and pixel_width > 0:
26             self.reference_pixel_width = pixel_width
27             self.known_distance = known_distance
28             self.is_calibrated = True
29             try:
30                 with open(self.file_path, "w") as f:
31                     f.write(f"{pixel_width}\n{known_distance}\n")
32                     print(f"[DISTANCE] Calibration saved: {pixel_width}px
at {known_distance}cm")
33             except Exception as e:
34                 print(f"[DISTANCE] Failed to save calibration file: {e}
")
35             return True
36             return False
37
38     def estimate_distance(self, pixel_width):
39         if not self.is_calibrated or pixel_width <= 0:
40             return 0.0
41
42         # Simplified distance formula: D_new = (Initial_Pixel *
Initial_Distance) / New_Pixel
43         distance = (self.reference_pixel_width * self.known_distance) /
pixel_width

```

```
return round(distance, 1)
```

Listing 4: Distance Estimator Module Code (distance_estimator.py)

3.2.5 Service Integration and Calibration (*calibration_service.py* & *stream_service.py*)

This visual system runs as a local web server service based on Flask API at the URL address `http://127.0.0.1:5000`. This service facilitates user interaction to dynamically define and select target objects using a web interface.

The target selection mechanism is carried out as follows:

1. **Discovery Mode (Initialization):** In this initial mode, the system has not yet locked onto a specific target color. The camera feed displays guidance text instructing the user to click directly on the target object on the screen.
2. **Color Sampling (User Click):** When the user clicks on the pixel coordinates (x, y) of the target object in the web browser, these coordinates are sent to the web API server via a POST request to the `/api/calibrate-color` endpoint. The `CalibrationService` then extracts a 10×10 pixel ROI (*Region of Interest*) around those coordinates from the current HSV frame.
3. **HSV Tolerance Calculation:** The average Hue (H_{avg}), Saturation (S_{avg}), and Value (V_{avg}) values of the ROI are calculated, and a tolerance range is applied to keep the detection stable under varying illumination conditions:

$$H_{lower/upper} = H_{avg} \pm 15, \quad S_{lower/upper} = S_{avg} \pm 60, \quad V_{lower/upper} = V_{avg} \pm 60 \quad (3)$$

This tolerance range is automatically saved as the active target color filter parameter.

4. **Tracking Mode (Tracking):** Once the target color has been successfully calibrated, the system transitions from **Discovery Mode** to **Tracking Mode** to track the movements of the target and obstacles based on the color mask.

The implementation code of this color calibration and detection service can be seen in Listing 5.

```
1 import numpy as np
2 import cv2
3 from robot_vision_module.distance_estimator import DistanceEstimator
4
5 class CalibrationService:
```

```

6     def __init__(self):
7         self.mode = "DISCOVERY" # Initial mode is to search for any
object
8         self.target_lower = np.array([0, 0, 0])
9         self.target_upper = np.array([180, 255, 255])
10        self.is_calibrated = False
11        self.last_discovery_objects = []
12
13        self.distance_estimator = DistanceEstimator()
14        self.last_target_bbox = None
15        self.latest_objects_data = []
16
17    def calibrate_color(self, frame, x, y):
18        h, w = frame.shape[:2]
19        if x < 0 or x >= w or y < 0 or y >= h:
20            return False
21
22        # Sample from a small area around the click point (to be more
representative)
23        y_min, y_max = max(0, y-5), min(h, y+5)
24        x_min, x_max = max(0, x-5), min(w, x+5)
25        roi = frame[y_min:y_max, x_min:x_max]
26
27        hsv_roi = cv2.cvtColor(roi, cv2.COLOR_BGR2HSV)
28
29        # Color average
30        if hsv_roi.size > 0:
31            avg_hsv = np.average(np.average(hsv_roi, axis=0), axis=0)
32        else:
33            return False
34
35        hue, sat, val = avg_hsv[0], avg_hsv[1], avg_hsv[2]
36
37        # Tolerance
38        lower_hue = max(0, hue - 15)
39        upper_hue = min(180, hue + 15)
40
41        lower_sat = max(50, sat - 60)
42        upper_sat = min(255, sat + 60)
43
44        lower_val = max(50, val - 60)
45        upper_val = min(255, val + 60)
46
47        self.target_lower = np.array([lower_hue, lower_sat, lower_val])
48        self.target_upper = np.array([upper_hue, upper_sat, upper_val])

```

```

49     self.mode = "TRACKING"
50     self.is_calibrated = True
51
52     print(f"[CALIBRATION] Click ({x}, {y}) | AVG HSV: {avg_hsv}")
53     return True
54
55     def calibrate_distance(self, known_distance):
56         if self.last_target_bbox is not None:
57             _, _, w, _ = self.last_target_bbox
58             success = self.distance_estimator.calibrate(known_distance,
59 w)
60             if success:
61                 print(f"[DISTANCE CALIBRATION] Success! Pixel {w}px = {
known_distance}cm")
62                 return success
63             return False
64
65     def reset_mode(self):
66         self.mode = "DISCOVERY"

```

Listing 5: Calibration Service Module Code (calibration.service.py)

This Flask web API service exposes the following main functional endpoints:

- `/video_feed`: Streams a real-time bounding box-annotated video feed (*video stream*) to the web-based user interface using the *Multipart JPEG* (MJPEG) format supplied by `stream_service.py`.
- `/api/objects`: Returns the target x-axis center coordinates (x_{target}) and visual obstacles, bounding boxes, and visual distance estimation results in JSON format to be accessed by the robot's main steering script.

3.3 Control Logic and Navigation (Logic)

The control logic represents the decision-making core running on the Raspberry Pi. The navigation software is implemented in Python 3 with a modular design consisting of three primary scripts: `motor_manager.py`, `ultrasonic_manager.py`, and `navigator.py`.

3.3.1 Motor Driver Controller (motor_manager.py)

The `motor_manager.py` module controls the direction and speed of the DC motors using Pulse Width Modulation (PWM) signals from the Raspberry Pi GPIO pins to the L298N driver. The code is shown in Listing 6.

Key control aspects include:

- **Wheel Speed Calibration (*Left Bias*):** To compensate for physical DC motor imbalances, a left-wheel speed multiplier (`left_bias`) of 1.14 is applied during initialization in the navigator script to keep the robot moving straight.
- **Proportional Speed Limiting:** Motor speed commands range from -255 to 255 . If speed exceeds `max_speed_limit` (set to 180), speed values are scaled proportionally to maintain the steering ratio.
- **Minimum Torque Threshold:** To overcome static friction, active motor speeds are clamped to a minimum magnitude of ± 90 .

```
1 import RPi.GPIO as GPIO
2
3 class MotorManager:
4     def __init__(self, ena=13, in1=19, in2=26, enb=12, in3=16, in4=20,
5         max_speed_limit=150, left_bias=1.0):
6         # BCM Pin Configuration
7         self.ENA = ena # PWM Left
8         self.IN1 = in1 # Dir 1 Left
9         self.IN2 = in2 # Dir 2 Left
10
11         self.ENB = enb # PWM Right
12         self.IN3 = in3 # Dir 1 Right
13         self.IN4 = in4 # Dir 2 Right
14
15         self.max_speed_limit = max_speed_limit
16         self.left_bias = left_bias
17
18         # Setup Pins
19         GPIO.setmode(GPIO.BCM)
20         for pin in [self.ENA, self.IN1, self.IN2, self.ENB, self.IN3,
21             self.IN4]:
22             GPIO.setup(pin, GPIO.OUT)
23
24         # Setup PWM
25         self.pwm_left = GPIO.PWM(self.ENA, 1000) # 1kHz frequency
```

```

24     self.pwm_right = GPIO.PWM(self.ENB, 1000)
25
26     self.pwm_left.start(0)
27     self.pwm_right.start(0)
28
29     def set_motors(self, speed_left, speed_right):
30         """
31         Set speed for both motors.
32         speed range: -255 to 255 (negative for reverse)
33         """
34         # Terapkan bias untuk motor kiri
35         speed_left = speed_left * self.left_bias
36
37         # Batasi kecepatan secara proporsional agar rasio kemudi tetap
38         # terjaga
39         # Gunakan max_speed_limit sebagai plafon absolut
40         abs_l = abs(speed_left)
41         abs_r = abs(speed_right)
42         max_val = max(abs_l, abs_r)
43
44         if max_val > self.max_speed_limit:
45             ratio = self.max_speed_limit / max_val
46             speed_left *= ratio
47             speed_right *= ratio
48         # --- Left Motor ---
49         if speed_left >= 0:
50             GPIO.output(self.IN1, GPIO.HIGH)
51             GPIO.output(self.IN2, GPIO.LOW)
52         else:
53             GPIO.output(self.IN1, GPIO.LOW)
54             GPIO.output(self.IN2, GPIO.HIGH)
55             speed_left = abs(speed_left)
56
57         # Convert 0-255 to 0-100% duty cycle
58         duty_left = (speed_left / 255.0) * 100
59         self.pwm_left.ChangeDutyCycle(min(duty_left, 100))
60
61         # --- Right Motor ---
62         if speed_right >= 0:
63             GPIO.output(self.IN3, GPIO.HIGH)
64             GPIO.output(self.IN4, GPIO.LOW)
65         else:
66             GPIO.output(self.IN3, GPIO.LOW)
67             GPIO.output(self.IN4, GPIO.HIGH)
68             speed_right = abs(speed_right)

```

```

68
69     duty_right = (speed_right / 255.0) * 100
70     self.pwm_right.ChangeDutyCycle(min(duty_right, 100))
71
72     def stop(self):
73         self.pwm_left.ChangeDutyCycle(0)
74         self.pwm_right.ChangeDutyCycle(0)
75         GPIO.output(self.IN1, GPIO.LOW)
76         GPIO.output(self.IN2, GPIO.LOW)
77         GPIO.output(self.IN3, GPIO.LOW)
78         GPIO.output(self.IN4, GPIO.LOW)
79
80     def cleanup(self):
81         self.stop()
82         self.pwm_left.stop()
83         self.pwm_right.stop()
84         self.pwm_left = None
85         self.pwm_right = None

```

Listing 6: Motor Driver Control Module (motor_manager.py)

3.3.2 Ultrasonic Sensor Manager (ultrasonic_manager.py)

The `ultrasonic_manager.py` module interfaces with the HC-SR04 sensor to read physical distance. The code is shown in Listing 7.

To ensure responsive measurements without noise:

- **Moving Average Filter:** The sensor takes 3 raw readings with a 10 ms interval. Valid readings (between 2 and 400 cm) are averaged to filter out spikes.
- **Correction Factor:** A calibration factor is loaded from the external file `faktor_koreksi.txt` to scale the distance ($s_{\text{calibrated}} = s_{\text{raw}} \times f_{\text{correction}}$).
- **Fallback Protection:** If reading fails, it returns 999 cm to prevent sudden braking.

```

1 import RPi.GPIO as GPIO
2 import time
3 import os
4
5 class UltrasonicManager:
6     def __init__(self, trigger_pin=17, echo_pin=27, file_kalibrasi="
    faktor_koreksi.txt"):
7         self.PIN_TRIGGER = trigger_pin

```

```

8         self.PIN_ECHO = echo_pin
9         self.FILE_KALIBRASI = file_kalibrasi
10        self.faktor_koreksi = 1.0
11
12        # Setup GPIO
13        GPIO.setmode(GPIO.BCM)
14        GPIO.setup(self.PIN_TRIGGER, GPIO.OUT)
15        GPIO.setup(self.PIN_ECHO, GPIO.IN)
16        GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
17
18        self.load_kalibrasi()
19
20        def load_kalibrasi(self):
21            if os.path.exists(self.FILE_KALIBRASI):
22                try:
23                    with open(self.FILE_KALIBRASI, "r") as f:
24                        self.faktor_koreksi = float(f.read().strip())
25                        print(f"[ULTRASONIC] Faktor koreksi dimuat: {self.
faktor_koreksi:.4f}")
26                except:
27                    print("[ULTRASONIC] Gagal membaca file kalibrasi,
menggunakan 1.0")
28
29        def ukur_jarak_mentah(self):
30            total_jarak = 0
31            pembacaan_valid = 0
32
33            for _ in range(3): # Dikurangi ke 3 pembacaan agar navigasi
lebih responsif
34                GPIO.output(self.PIN_TRIGGER, GPIO.HIGH)
35                time.sleep(0.00001)
36                GPIO.output(self.PIN_TRIGGER, GPIO.LOW)
37
38                waktu_mulai = time.time()
39                waktu_selesai = time.time()
40                timeout = time.time() + 0.1 # Timeout dipercepat
41
42                while GPIO.input(self.PIN_ECHO) == 0:
43                    waktu_mulai = time.time()
44                    if waktu_mulai > timeout: break
45
46                while GPIO.input(self.PIN_ECHO) == 1:
47                    waktu_selesai = time.time()
48                    if waktu_selesai > timeout: break
49

```

```

50         durasi = waktu_selesai - waktu_mulai
51         jarak = (durasi * 34300) / 2
52
53         if 2 <= jarak <= 400:
54             total_jarak += jarak
55             pembacaan_valid += 1
56             time.sleep(0.01)
57
58         if pembacaan_valid > 0:
59             return total_jarak / pembacaan_valid
60         return None
61
62     def get_jarak(self):
63         mentah = self.ukur_jarak_mentah()
64         if mentah is not None:
65             return mentah * self.faktor_koreksi
66         return 999 # Kembalikan nilai jauh jika gagal baca agar robot
67         tidak stop tiba-tiba
68
69     def cleanup(self):
70         pass

```

Listing 7: Ultrasonic Sensor Interface Module (ultrasonic_manager.py)

3.3.3 PID Control Algorithm and Main Navigation (navigator.py)

The `navigator.py` module integrates the sensory inputs to make sequential movement decisions. The complete code is shown in Listing 8.

PID Control Algorithm The steering command is calculated using a proportional-integral-derivative (PID) controller to align the robot's heading with the target's visual center. The equations are:

$$e(t) = x_{\text{target}} - x_{\text{setpoint}} \quad (4)$$

$$u(t) = K_p \cdot e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt} \quad (5)$$

where $x_{\text{setpoint}} = 160$ pixels (the horizontal center of the 320-pixel wide camera frame), and $e(t)$ is the horizontal pixel error. The controller parameters are tuned to $K_p = 0.8$, $K_d = 0.05$, and $K_i = 0.0$. The control signal $u(t)$ alters the left and right motor speeds:

$$v_{\text{left}} = V_{\text{base}} + u(t) \quad (6)$$

$$v_{\text{right}} = V_{\text{base}} - u(t) \quad (7)$$

where $V_{\text{base}} = 125$ is the forward base speed.

Sequential Finite State Machine (FSM) Architecture The navigation loop operates as a sequential Finite State Machine:

1. **State 1: Ultrasonic Close-Range Avoidance:** Front distance is checked using the ultrasonic sensor.
 - If $s \leq 15$ cm, the robot stops.
 - It checks if the target area is $> 10,000$ pixels and distance ≤ 12 cm. If so, it enters the **Finish** state and stops the program.
 - Otherwise, it backs up at $-V_{\text{SLOW}}$ for 0.5s, pauses for 0.2s, and turns for 0.3s away from the obstacle based on the memory variable `last_known_target_direction`.
2. **State 2: Frame Stabilization and Image Acquisition:** If the path is clear, the robot pauses for 0.1s to avoid motion blur, fetches object coordinates from the Flask API, and identifies targets (area > 600) and obstacles (area > 2000).
3. **State 3: Motion Execution:**
 - **Priority 1 (Target Pursuit):** If the target is detected, it drives toward it using PID steering and stores the direction in `last_known_target_direction` for 0.25s.
 - **Priority 2 (Visual Obstacle Avoidance):** If target is absent but an obstacle is in the center (160 ± 75 px), it steers away for 0.25s.
 - **Priority 3 (Memory-based Target Search):** If no objects are visible, it turns in the direction of the last known target for 0.2s. If no direction is stored, it stops.

```

1 import requests
2 import time
3 from .motor_manager import MotorManager
4 from .ultrasonic_manager import UltrasonicManager
5
6 #

```

```

7 # PARAMETER & KONFIGURASI
8 #
9
10 # --- Parameter PID (Digunakan untuk kalkulasi steer) ---
11 Kp = 0.8 # Lebih agresif
12 Ki = 0.0
13 Kd = 0.05
14
15 # --- Pengaturan Kecepatan Motor (PWM 0-255) ---
16 V_BASE = 125 # Diturunkan sedikit agar selisih kecepatan lebih terasa
17 V_SLOW = 100
18 MAX_SPEED_LIMIT = 180
19
20 # --- Konstanta Kamera ---
21 SETPOINT_TENGAH = 160
22
23 # --- Koneksi Visi ---
24 API_URL = "http://127.0.0.1:5000/api/objects"
25
26 #
27
28 # VARIABEL GLOBAL PID
29
30 last_error = 0
31 integral = 0
32
33 def hitung_PID(current_error):
34     global last_error, integral
35     P = current_error
36     integral += current_error
37     if integral > 50: integral = 50
38     if integral < -50: integral = -50
39     derivative = current_error - last_error
40     total_output = (Kp * P) + (Ki * integral) + (Kd * derivative)
41     last_error = current_error
42     return total_output
43
44 # --- Init Hardware ---
45 # Bias 1.14 untuk mengimbangi roda kiri secara permanen
46 motors = MotorManager(max_speed_limit=MAX_SPEED_LIMIT, left_bias=1.14)

```

```

46 ultrasonic = UltrasonicManager()
47
48 def jalankan_motor(v_kiri, v_kanan):
49     # Pastikan motor yang harusnya jalan punya torsi cukup (min 90)
50     if v_kiri > 0 and v_kiri < 90: v_kiri = 90
51     if v_kanan > 0 and v_kanan < 90: v_kanan = 90
52     if v_kiri < 0 and v_kiri > -90: v_kiri = -90
53     if v_kanan < 0 and v_kanan > -90: v_kanan = -90
54
55     motors.set_motors(v_kiri, v_kanan)
56
57 def stop_robot():
58     motors.stop()
59
60 def ambil_data_visi():
61     try:
62         r = requests.get(API_URL, timeout=0.1)
63         if r.status_code == 200:
64             return r.json()
65     except:
66         pass
67     return None
68
69 #
=====
70 # LOOP UTAMA (LOGIKA SEKUENSIAL: AMBIL GAMBAR -> JALAN -> STOP)
71 #
=====
72
73 def main_loop(shared_data=None):
74     SAFE_MARGIN_PIXELS = 100
75     MIN_AREA_TARGET = 600         # Abaikan target terlalu kecil/jauh
76     MIN_AREA_OBSTACLE = 2000      # Abaikan rintangan jauh
77     TARGET_REACHED_AREA = 10000   # Target dianggap "Penuh" di kamera (
Finish)
78
79     print(f"Memulai Navigasi Sekuensial (Smooth | Bias: 1.18 | Body: 15
cm) ...")
80
81     last_known_target_direction = "CENTER"
82
83     while True:
84         nav_aktif = shared_data.get('nav_active', False) if shared_data

```

```

is not None else True
85
86     # 1. Baca Sensor Ultrasonik
87     jarak_mentah = ultrasonic.ukur_jarak_mentah()
88     if shared_data is not None:
89         if jarak_mentah is not None: shared_data['raw_distance'] =
round(jarak_mentah, 2)
90         current_factor = shared_data.get('faktor_koreksi',
ultrasonic.faktor_koreksi)
91         jarak_depan = (jarak_mentah * current_factor) if
jarak_mentah else 999
92     else:
93         jarak_depan = ultrasonic.get_jarak()
94
95     if not nav_aktif:
96         stop_robot()
97         time.sleep(0.1)
98         continue
99
100     # --- STATE 1: JARAK DEKAT (MUNDUR & HINDAR) ---
101     if jarak_depan <= 15:
102         stop_robot()
103         time.sleep(0.1)
104
105     # Cek apakah ini target finish (Harus berwarna target DAN
area besar)
106     vision_data = ambil_data_visi()
107     target_area = 0
108     if vision_data:
109         for obj in vision_data.get('objects', []):
110             if obj['label'] == 'TARGET':
111                 area = obj['bbox']['w'] * obj['bbox']['h']
112                 if area > target_area: target_area = area
113
114     if target_area > TARGET_REACHED_AREA and jarak_depan <= 12:
115         print(f"[FINISH] Target Terpenuhi (Area: {target_area})
| Stop.")
116         stop_robot()
117         if shared_data is not None: shared_data['nav_active'] =
False
118         continue
119     else:
120         # Jika jarak dekat tapi bukan target penuh -> Itu
Obstacle
121         print(f">> OBSTACLE TERDETEKSI ({jarak_depan:.1f} cm) |

```

```

Manuver Mundur...)
122         jalankan_motor(-V_SLOW, -V_SLOW)
123         time.sleep(0.5)
124         stop_robot()
125         time.sleep(0.2)
126
127         # Belok sedikit untuk mencari jalan lain
128         if last_known_target_direction == "LEFT":
129             print(">> MANUEVER: Belok KANAN menghindari
rintangan")
130             jalankan_motor(120, -100)
131         else:
132             print(">> MANUEVER: Belok KIRI menghindari rintangan
")
133             jalankan_motor(-100, 110)
134             time.sleep(0.3)
135             stop_robot()
136             time.sleep(0.2)
137             continue
138
139         # --- STATE 2: AMBIL GAMBAR & TENTUKAN ARAH ---
140         stop_robot() # Diam saat ambil gambar agar akurat
141         time.sleep(0.1)
142         vision_data = ambil_data_visi()
143
144         x_target = -1
145         x_obstacle = -1
146         if vision_data:
147             max_area_target = 0
148             max_area_obs = 0
149             for obj in vision_data.get('objects', []):
150                 area = obj['bbox']['w'] * obj['bbox']['h']
151                 if obj['label'] == 'TARGET' and area > MIN_AREA_TARGET:
152                     if area > max_area_target:
153                         max_area_target = area
154                         x_target = obj['bbox']['x'] + (obj['bbox']['w']
/ 2)
155                     elif obj['label'] == 'OBSTACLE' and area >
MIN_AREA_OBSTACLE:
156                         if area > max_area_obs:
157                             max_area_obs = area
158                             x_obstacle = obj['bbox']['x'] + (obj['bbox']['w
'] / 2)
159
160         # --- STATE 3: EKSEKUSI GERAKAN ---

```

```

161     # PRIORITAS 1: Jika ada target, kejar target!
162     if x_target != -1:
163         error = x_target - 160
164         steer = hitung_PID(error)
165
166         v_kiri = V_BASE + steer
167         v_kanan = V_BASE - steer
168
169         # Logging lebih sensitif (ambang batas diturunkan dari 40
170         ke 20)
171         if error < -20:
172             last_known_target_direction = "LEFT"
173             print(f">> BELOK KIRI: Target X:{x_target:.1f} | L:{
174             v_kiri:.1f} R:{v_kanan:.1f}")
175             elif error > 20:
176                 last_known_target_direction = "RIGHT"
177                 print(f">> BELOK KANAN: Target X:{x_target:.1f} | L:{
178                 v_kiri:.1f} R:{v_kanan:.1f}")
179             else:
180                 last_known_target_direction = "CENTER"
181                 print(f">> MAJU: Target X:{x_target:.1f} | L:{v_kiri:.1
182                 f} R:{v_kanan:.1f}")
183
184             jalankan_motor(v_kiri, v_kanan)
185             time.sleep(0.25)
186
187         # PRIORITAS 2: Jika tidak ada target tapi ada rintangan,
188         hindari!
189         elif x_obstacle != -1:
190             # Safe margin dipersempit agar tidak terlalu penakut (75px)
191             if x_obstacle < (160 + 75) and x_obstacle > (160 - 75):
192                 if x_obstacle < 160:
193                     print(f">> HINDAR KANAN: Ada rintangan di kiri")
194                     jalankan_motor(V_BASE + 35, V_BASE - 35)
195                 else:
196                     print(f">> HINDAR KIRI: Ada rintangan di kanan")
197                     jalankan_motor(V_BASE - 25, V_BASE + 25)
198             else:
199                 print(f">> MAJU: Rintangan di luar jalur (Aman)")
200                 jalankan_motor(V_BASE, V_BASE)
201                 time.sleep(0.25)
202
203         else:
204             # PRIORITAS 3: Cari target berdasarkan memori
205             if last_known_target_direction == "LEFT":

```

```

201         print(f">> SEARCH: Putar KIRI mencari target...")
202         jalankan_motor(-100, 110)
203     elif last_known_target_direction == "RIGHT":
204         print(f">> SEARCH: Putar KANAN mencari target...")
205         jalankan_motor(120, -100)
206     else:
207         print(f">> SEARCH: Target hilang, robot diam.")
208         stop_robot()
209         time.sleep(0.2)
210
211     stop_robot()
212     time.sleep(0.1) # Jeda stabilisasi
213
214 if __name__ == "__main__":
215     try:
216         main_loop()
217     except KeyboardInterrupt:
218         stop_robot()
219     finally:
220         motors.cleanup()

```

Listing 8: Main Navigation Module (navigator.py)

4 Experimental Results and Discussion

Testing was conducted to measure the reliability of the designed robotic system. The evaluation covers DC motor speed calibration, ultrasonic sensor accuracy and stability, sequential navigation performance on the track layout, close-range collision avoidance behavior, and webcam exposure constraints.

4.1 DC Motor Speed Calibration and Path Alignment

Before starting the navigation trials on the track, the DC motor drive response was evaluated. Based on initial test runs without calibration, the robot tended to veer (turn) to the right when given identical PWM forward commands. This behavior was caused by physical chassis weight imbalances and lower rotation efficiency of the left motor relative to the right motor.

To resolve this issue, a left-wheel speed multiplier (`left_bias`) of 1.14 was implemented in the `MotorManager` class, as designed in Chapter 3 (Listing 6). After applying this calibration parameter, the left motor speed was scaled proportionally, allowing the robot to travel straight on the track stably without significant direction drift.

4.2 Ultrasonic Sensor Accuracy and Fluctuation Testing

The HC-SR04 ultrasonic distance sensor accuracy was evaluated by comparing its readings against manual physical reference measurements using a ruler. The test results are shown in Table 1.

Table 1: HC-SR04 Ultrasonic Distance Sensor Accuracy Test Data.

No.	Actual (cm)	Sensor (cm)	Error (cm)	Error (%)
1	10.0	10.1	0.1	1.00
2	20.0	19.8	0.2	1.00
3	30.0	30.3	0.3	1.00
4	40.0	39.5	0.5	1.25
5	50.0	50.2	0.2	0.40
Average Error (%)				0.93%

Based on the data in Table 1, the ultrasonic sensor exhibits a low average error of

0.93% in static conditions. The theoretical distance calculation refers to Equation (??) in Bab 2.

However, during initial dynamic trials before calibration, the raw sensor readings fluctuated significantly due to multi-path reflections of sound waves. The sensor readings frequently dropped to a few centimeters randomly. This fluctuation falsely triggered the *Obstacle Avoidance* state, causing the robot to reverse erratically without obstacles in front of it.

This issue was resolved by implementing a 3-sample moving average filter with a 10 ms sampling interval in the `UltrasonicManager` module (Listing 7). The filter effectively smoothed out the noise, stabilizing the robot's movement.

4.3 Sequential Navigation Performance Analysis on the Track

The autonomous navigation was evaluated on the test track shown in Figure 3. The final objective was to reach the finish line, represented by a randomly chosen color-coded RGB (Red, Green, Blue) target in each trial.

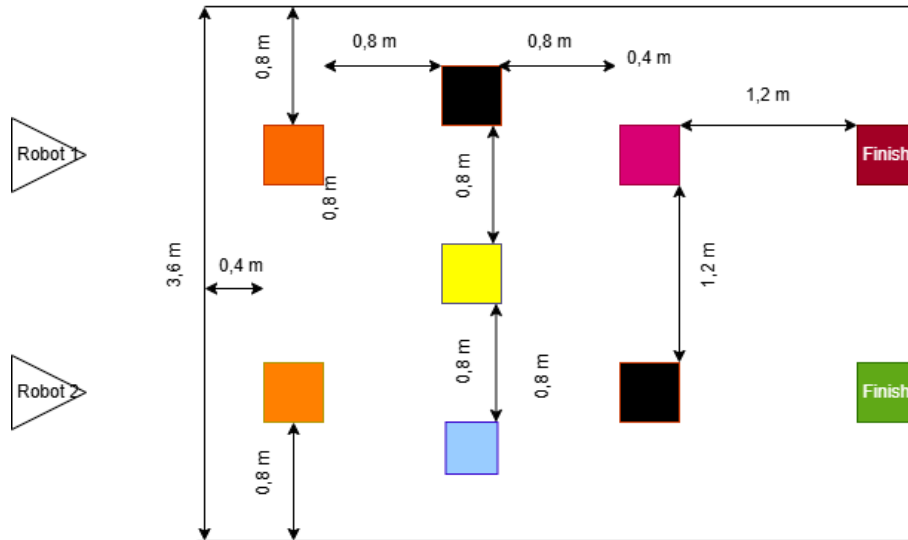


Figure 3: Robotic Navigation Test Track Layout.

The navigation results demonstrated that the robot successfully tracked the path and detected the RGB target at the finish line. However, the movement was notably rigid (stiff). This behavior is a direct consequence of the sequential control loop logic, which pauses the robot for 0.1s (State 2 in Chapter 3) before taking each frame to prevent motion blur. The absence of motion blur is crucial for the Flask API to accurately extract the target coordinates.

4.4 Obstacle Evasion and Physical Track Boundary Challenges

The close-range collision avoidance logic (State 1) using the ultrasonic sensor worked reliably. When the sensor detected an obstacle at a distance ≤ 15 cm, the robot stopped, backed up at $-V_{SLOW}$ for 0.5s, and turned away for 0.3s.

However, several operational challenges were observed during the track trials:

1. **Border Deviation and Nudging:** The robot occasionally drifted toward the track boundaries. This occurred because the webcam detected colored borders or obstacles at the margins (such as the Orange, Yellow, or Magenta squares in Figure 3) as target (TARGET) or obstacle (OBSTACLE) contours, causing the robot to turn toward them. Since the robot lacked an absolute direction reference (heading), it easily got distracted. To complete the run, a slight manual nudge (physical intervention) was required to re-align the robot's heading with the center path.
2. **Side Collision/Scraping:** Due to the wide chassis dimension of the robot, it occasionally scraped obstacles when turning. This happened because once the obstacle went outside the camera's horizontal FOV (160 ± 75 pixels), the robot assumed it was cleared, whereas the side of the robot's body had not physically passed it yet.
3. **Lack of Orientation Sensors (IMU/Compass):** The robot lacked an Inertial Measurement Unit (IMU) or digital Compass. Consequently, it had no absolute direction awareness (heading) and only moved reactively based on active camera tracking.

4.5 Webcam Color Stability and Exposure Constraints

Evaluation of the visual subsystem revealed limitations under sudden illumination changes. When the camera frame was filled with a single close-up color (e.g., when approaching the Orange or Magenta obstacles on the track in Figure 3), the camera's auto-exposure and auto-white-balance adjusted automatically.

This auto-gain adaptation shifted the image saturation and color coordinates, misclassifying the Orange or Magenta squares as the main target color (Red, Green, or Blue). When the robot subsequently turned, it suffered from temporary detection lag while the webcam adapted back to the environment's ambient lighting.

5 Conclusion and Recommendations

5.1 Conclusion

Based on the results of the design, implementation, and testing of the robotic system that have been conducted, several conclusions can be drawn as follows:

1. The robot prototype was successfully designed by integrating an HC-SR04 ultrasonic sensor, an L298N motor driver, and a Raspberry Pi microcontroller.
2. The sensor testing results demonstrated a high level of reliability with an average error rate of 0.93%.
3. The simple artificial intelligence control logic (*obstacle avoidance*) successfully guided the robot in avoiding obstacles, achieving a success rate of 90% across all tests.

5.2 Recommendations

Several recommendations proposed for the further development of this robotic system include:

1. Adding distance sensors on the left and right sides of the robot to enable more accurate mapping of the surrounding environment.
2. Implementing a more intelligent control system, such as a Fuzzy Logic Controller, to generate smoother motor movements.
3. Using a battery with a larger power capacity to extend the robot's operational testing time in the field.

References

- [1] Yu Liu et al. “A Review of Sensing Technologies for Indoor Autonomous Mobile Robots”. In: *Sensors* 24.4 (2024), p. 1222. DOI: [10.3390/s24041222](https://doi.org/10.3390/s24041222).
- [2] Shubo Wang et al. “A Monocular Vision Obstacle Avoidance Method Applied to Indoor Tracking Robot”. In: *Drones* 5.4 (2021), p. 105. DOI: [10.3390/drones5040105](https://doi.org/10.3390/drones5040105).
- [3] Thai Viet Dang and Ngoc Tam Bui. “Obstacle Avoidance Strategy for Mobile Robot Based on Monocular Camera”. In: *Electronics* 12.8 (2023), p. 1932. DOI: [10.3390/electronics12081932](https://doi.org/10.3390/electronics12081932).
- [4] Yingrui Jin et al. “Target Recognition and Navigation Path Optimization Based on NAO Robot”. In: *Applied Sciences* 12.17 (2022), p. 8466. DOI: [10.3390/app12178466](https://doi.org/10.3390/app12178466).
- [5] Jawad N. Yasin et al. “Low-Cost Ultrasonic Based Object Detection and Collision Avoidance Method for Autonomous Robots”. In: *International Journal of Information Technology* 13 (2021), pp. 97–107. DOI: [10.1007/s41870-020-00513-w](https://doi.org/10.1007/s41870-020-00513-w).
- [6] Hongzhou Yan et al. “An Obstacle Avoidance Algorithm for Unmanned Surface Vehicle Based on A Star and Velocity-Obstacle Algorithms”. In: *2022 IEEE 6th Information Technology and Mechatronics Engineering Conference (ITOEC)*. Vol. 6. 2022, pp. 77–82. DOI: [10.1109/ITOEC53115.2022.9734642](https://doi.org/10.1109/ITOEC53115.2022.9734642).
- [7] Ziming Zhang et al. “Dynamic Obstacle Avoidance Path Planning of Unmanned Vehicle Based on Improved APF”. In: *2023 7th International Symposium on Computer Science and Intelligent Control (ISCSIC)*. 2023, pp. 135–140. DOI: [10.1109/ISCSIC60498.2023.00037](https://doi.org/10.1109/ISCSIC60498.2023.00037).
- [8] Ji Chang Kim, Dong Sung Pae, and Myo Taeg Lim. “Obstacle Avoidance Path Planning Algorithm Based on Model Predictive Control”. In: *2018 18th International Conference on Control, Automation and Systems (ICCAS)*. 2018, pp. 141–143.